

**Universidade Federal da Fronteira Sul - UFFS**  
**Ciência da Computação**  
**Estrutura de Dados II**  
**Guilherme Dal Bianco**

Niumar Girardi

**Relatório Técnico de Implementação: Estrutura de Dados Árvore B+**  
[https://github.com/NewtNiu/Estrutura\\_de\\_Dados-Arvore\\_BPlus](https://github.com/NewtNiu/Estrutura_de_Dados-Arvore_BPlus)

**Chapecó**  
**28/11/2025**

## Introdução

A Árvore B+ é uma estrutura de dados amplamente utilizada em sistemas de banco de dados, sistemas de arquivos e mecanismos de indexação que envolvem grande volume de informações. Assim como a Árvore B tradicional, trata-se de uma estrutura balanceada, garantindo buscas, inserções e remoções eficientes, geralmente em tempo  $O(\log n)$ .

Sua principal diferença em relação à Árvore B clássica é que todas as chaves buscáveis estão armazenadas exclusivamente nas folhas, enquanto os nós internos funcionam apenas como “separadores” para guiar a navegação. Além disso, suas folhas são encadeadas sequencialmente, permitindo iteração ordenada direta, um fator crítico para sistemas que realizam leituras sequenciais em disco, como SGBDs.

Este relatório apresenta:

Sumario:

|                                           |           |
|-------------------------------------------|-----------|
| <b>Objetivos</b>                          | <b>3</b>  |
| <b>Fundamentação Teórica</b>              | <b>4</b>  |
| Conceito de Árvore B+:                    | 4         |
| Ordem da Árvore:                          | 4         |
| Navegação e Encadeamento das Folhas       | 4         |
| <b>Desenvolvimento e Implementação</b>    | <b>5</b>  |
| Arquitetura Geral                         | 5         |
| Estrutura do Nó:                          | 5         |
| Estrutura da Árvore                       | 5         |
| Arquivo main.c                            | 6         |
| Arquivo bptree.h                          | 6         |
| Arquivo bptree.c - Implementação Completa | 6         |
| Criação e Navegação                       | 7         |
| Busca                                     | 7         |
| Inserção                                  | 7         |
| Remoção                                   | 8         |
| Impressão da Árvore                       | 9         |
| Resultados e Utilização                   | 9         |
| <b>Conclusão</b>                          | <b>15</b> |

## Objetivos

O objetivo deste trabalho é desenvolver uma compreensão sólida sobre o funcionamento de uma Árvore B+ de ordem 4 por meio da análise, implementação e experimentação direta dessa estrutura de dados. Para isso, busca-se construir e executar uma implementação funcional em linguagem C, explorando todas as operações fundamentais realizadas por esse tipo de árvore.

Entre essas operações estão a inserção, busca e remoção de chaves, permitindo observar na prática como a estrutura mantém sua organização e balanceamento. Também pretende-se analisar o comportamento interno da árvore durante eventos críticos, como divisões de nós (split), fusões (merge) e empréstimos de chaves (borrow), compreendendo como a árvore ajusta sua estrutura para preservar suas propriedades.

Além disso, o trabalho visa documentar detalhadamente o funcionamento da implementação, descrevendo as estruturas utilizadas, a lógica interna das funções e os resultados obtidos durante os testes. Dessa forma, busca-se tanto consolidar o conhecimento teórico quanto demonstrar a correta aplicação dos princípios que regem as Árvores B+.

## Fundamentação Teórica

### Conceito de Árvore B+:

A Árvore B+ é uma árvore de busca balanceada de alta capacidade. Seu objetivo principal é minimizar o número de acessos a disco, trabalhando com nós grandes e operações logarítmicas.

Suas principais características são:

- Todas as chaves buscáveis residem nas folhas.
- Nós internos atuam apenas como separadores.
- Cada nó pode armazenar vários valores (ordem definida pelo projeto).
- As folhas são encadeadas por meio de ponteiros “next”.
- A profundidade permanece baixa, mesmo com grandes volumes de dados.
- Inserções e remoções mantêm a árvore sempre balanceada.

### Ordem da Árvore:

```
// ORDER = 4 significa que cada nó pode ter até 4 filhos
#define ORDER 4
// Número máximo de chaves em um nó
#define MAX_KEYS (ORDER - 1)
// Número mínimo de chaves em um nó (após remoções)
#define MIN_KEYS ((MAX_KEYS + 1)/2)
```

Isso significa:

- Cada nó comporta no máximo 3 chaves.
- Cada nó interno possui até 4 filhos.
- Após remoção, um nó não pode ficar com menos de 2 chaves (exceto a raiz).

## Navegação e Encadeamento das Folhas

Durante uma busca:

1. O algoritmo desce pelos nós internos comparando separadores.
2. Ao alcançar as folhas, onde todas as chaves residem, realiza a busca final.

As folhas são encadeadas assim:

[f1] -> [f2] -> [f3] -> NULL

Garantindo iteração rápida e ordenada.

## Desenvolvimento e Implementação

A implementação foi estruturada em três arquivos principais:

- **main.c** - interface e menu do usuário
- **bptree.h** - estruturas e protótipos
- **bptree.c** - lógica completa da Árvore B+

A seguir, cada parte é detalhada.

## Arquitetura Geral

### Estrutura do Nó:

```
// Estrutura que representa um nó na árvore B+
typedef struct BPTreeNode {
    int keys[MAX_KEYS + 1];           // Vetor de chaves
    armazenadas no nó
    struct BPTreeNode *children[ORDER + 1]; // Ponteiros para os filhos
    int numKeys;                       // Quantas chaves o nó
    possui atualmente
    bool isLeaf;                       // Diz se o nó é folha ou
    não
    struct BPTreeNode *next;           // Ponteiro para próxima
    folha (lista encadeada)
} BPTreeNode;
```

Componentes:

- **keys[]** - armazena chaves ordenadas.
- **children[]** - ponteiros para nós filhos (internos).
- **numKeys** - quantidade atual de chaves no nó.
- **isLeaf** - indica se o nó é folha.
- **next** - ponteiro para a próxima folha (encadeamento).

## Estrutura da Árvore

```
// Estrutura principal da Árvore B+
typedef struct {
    BPTreeNode *root; // Ponteiro para o nó raiz
} BPTree;
```

A árvore mantém apenas a raiz. Toda a navegação acontece a partir dela.

## Arquivo *main.c*

O arquivo **main.c** implementa a interface de usuário e as operações básicas:

Funções principais:

- **mostrarMenu()** - exibe as opções disponíveis
- **inserirValores()**
  - Inserção automática dos valores padrões solicitados no enunciado
  - Inserção manual de um valor único
- **buscarValor()** - utiliza a função **search()**
- **removerValor()** - utiliza **removeKey()**
- **printTree()** - exibe toda a árvore em pré-ordem e a lista de folhas

Fluxo geral do programa:

1. Criar a árvore (**createTree()**)
2. Exibir menu
3. Esperar entrada do usuário
4. Chamar o módulo correspondente

## Arquivo *bptree.h*

Define:

- Ordem da árvore
- Estrutura dos nós
- Estrutura da árvore
- Protótipos das funções públicas

Prototipadas:

```
// Protótipos das funções disponíveis para usar na árvore
BPTree *createTree();           // Cria uma árvore vazia
void insert(BPTree *tree, int key); // Insere uma chave
void removeKey(BPTree *tree, int key); // Remove uma chave
bool search(BPTree *tree, int key); // Procura uma chave
void printTree(BPTree *tree);    // Imprime a árvore completa
```

## Arquivo *bptree.c* - Implementação Completa

Este arquivo contém toda a lógica que faz a B+ Tree funcionar.  
A seguir, descrevemos cada grupo de funções.

### Criação e Navegação

#### **createNode(isLeaf)**

Cria um novo nó, inicializa vetores, ponteiros e marca se é folha.

#### **createTree()**

Inicializa a árvore criando uma raiz vazia marcada como folha.

#### **findLeaf(root, key)**

Desce pela árvore até encontrar a folha onde a chave deve estar/inserida/removida.

### Busca

#### **search(tree, key)**

Fluxo:

1. Encontra a folha com **findLeaf()**
2. Busca sequencial no vetor **keys[]**
3. Retorna verdadeiro/falso

A busca é  $O(\log n)$ , devido aos níveis da árvore.

### Inserção

As principais funções envolvidas:

#### **insertIntoLeafSorted(leaf, key)**

Insere na posição correta, mantendo ordenação.

### **splitLeaf(tree, leaf)**

Quando a folha fica com mais chaves que o limite:

- Divide o nó ao meio
- Cria uma nova folha
- Move metade das chaves para ela
- Encadeia via next
- Promove a primeira chave da nova folha

### **insertIntoParent(tree, left, promoted, right)**

Insere a chave promovida em um nó interno.

Se o nó interno também ficar cheio:

- É dividido
- A chave do meio sobe
- Pode gerar nova raiz

### **insert(tree, key)**

Fluxo completo de inserção:

- Buscar folha
- Inserir ordenadamente
- Verificar overflow
- Dividir se necessário
- Propagar splits até a raiz

## **Remoção**

Remoção é a parte mais complexa, incluindo:

### **removeFromLeaf(leaf, pos)**

Remove uma chave deslocando os elementos à esquerda.

### **Atualização de separadores**

Ao remover a menor chave de uma folha, o pai precisa atualizar sua chave correspondente.

### **Operações de Segurança (Borrow/Merge)**

Para evitar nós com menos chaves que o mínimo, a árvore utiliza:

Em folhas:

- **borrowFromLeftLeaf**
- **borrowFromRightLeaf**
- **mergeLeaves**

Em nós internos:

- **borrowFromLeftInternal**
- **borrowFromRightInternal**
- **mergeInternalChildren**

### **fixAfterDeletion(tree, node)**

Função que garante a manutenção das invariantes:

- MinKeys
- Ajuste de chaves do pai
- Empréstimo se possível
- Fusão quando necessário
- Correção recursiva até a raiz

## **Impressão da Árvore**

### **printTree(tree)**

Imprime:

- A árvore completa em pré-ordem
- A lista encadeada das folhas, por exemplo:

**[5 7] -> [12 17 20] -> [23 29 30] -> NULL**

Permitindo verificar rapidamente o estado final da B+ Tree após operações.

## **Resultados e Utilização**

Inserções utilizadas:

```
// Lista de valores pré-definidos
int valores[] = {10, 20, 5, 6, 12, 30, 7, 17, 29, 23, 12, 111,
10, 23};
```

Passo a passo do programa:

Após compilar:

```
PS D:\UFFS\TerceiroSemestre\Estrutura de dados II\T3_ArvoreBplus> gcc -std=c11 bptree.c main.c -o bptree
PS D:\UFFS\TerceiroSemestre\Estrutura de dados II\T3_ArvoreBplus> ./bptree

===== MENU B+ TREE =====
1 - Inserir valores
2 - Buscar valor
3 - Remover valor
4 - Ver arvore
0 - Sair
Escolha: █
```



Inserindo:

```
Escolha: 1

1 - Inserir valores padrao
2 - Inserir valor manualmente
Escolha: █
```

Escolhido valores padrões {10, 20, 5, 6, 12, 30, 7, 17, 29, 23, 12, 111, 10, 23}  
(E inserido por um for que chama a função `insert(tree, valores[i]);`)

Codigo:

```
void inserirValores(BPTree *tree) {

    int opc;

    // Mostra opções de inserção
    printf("\n1 - Inserir valores padrao\n");
    printf("2 - Inserir valor manualmente\n");
    printf("Escolha: ");

    if (scanf("%d", &opc) != 1) { while(getchar() != '\n'); return; }

    // Caso o usuário escolha inserir valores automáticos
    if (opc == 1) {
        // Lista de valores pré-definidos
        int valores[] = {10, 20, 5, 6, 12, 30, 7, 17, 29, 23, 12, 111, 10, 23};

        int n = sizeof(valores) / sizeof(int); // Calcula quantos valores têm no vetor

        printf("\nInserindo valores padrao:\n");
        // Percorre o vetor e insere cada valor na árvore
        for (int i = 0; i < n; i++) {
            printf("%d ", valores[i]); // Mostra o valor inserido
            insert(tree, valores[i]); // Chama a função de inserção
        }
        printf("\nValores inseridos com sucesso!\n");

        // Caso o usuário escolha inserir um valor manualmente
    } else if (opc == 2) {
        int v;
        printf("\nDigite o valor para inserir: ");
        if (scanf("%d", &v) != 1) { while(getchar() != '\n'); return; }

        insert(tree, v); // Insere o valor único
    }
}
```

```

        printf("\nValor %d inserido!\n", v);

    } else {
        printf("Opcao invalida\n");    // Caso o usuário digite um
número errado
    }
}

```

Saida:

```

Escolha: 1

Inserindo valores padrao:
10 20 5 6 12 30 7 17 29 23 12 111 10 23
Valores inseridos com sucesso!

===== MENU B+ TREE =====
1 - Inserir valores
2 - Buscar valor
3 - Remover valor
4 - Ver arvore
0 - Sair
Escolha: █

```

Verificando a arvore:

```

Escolha: 4

Arvore B+:
Nivel 0: [20]
Nivel 1: [10 12]
Nivel 2: [5 6 7]
Nivel 2: [10 10 12]
Nivel 2: [12 17]
Nivel 1: [29]
Nivel 2: [20 23 23]
Nivel 2: [29 30 111]

Folhas (encadeadas):
[5 6 7] -> [10 10 12] -> [12 17] -> [20 23 23] -> [29 30 111] -> NULL

===== MENU B+ TREE =====
1 - Inserir valores
2 - Buscar valor
3 - Remover valor
4 - Ver arvore
0 - Sair
Escolha: █

```

Buscas solicitadas 6 e 15:

6:

```
===== MENU B+ TREE =====
1 - Inserir valores
2 - Buscar valor
3 - Remover valor
4 - Ver arvore
0 - Sair
Escolha: 2

Digite o valor para buscar: 6

Valor 6 encontrado na arvore!
```

15:

```
===== MENU B+ TREE =====
1 - Inserir valores
2 - Buscar valor
3 - Remover valor
4 - Ver arvore
0 - Sair
Escolha: 2

Digite o valor para buscar: 15

Valor 15 NAO encontrado!

===== MENU B+ TREE =====
1 - Inserir valores
2 - Buscar valor
3 - Remover valor
4 - Ver arvore
0 - Sair
Escolha: █
```

Remoções solicitadas 6, 5 e 7:

6:

```
===== MENU B+ TREE =====
1 - Inserir valores
2 - Buscar valor
3 - Remover valor
4 - Ver arvore
0 - Sair
Escolha: 3

Digite o valor para remover: 6

Valor 6 removido!

===== MENU B+ TREE =====
1 - Inserir valores
2 - Buscar valor
3 - Remover valor
4 - Ver arvore
0 - Sair
Escolha: 4

Arvore B+:
Nivel 0: [20]
Nivel 1: [10 12]
Nivel 2: [5 7]
Nivel 2: [10 10 12]
Nivel 2: [12 17]
Nivel 1: [29]
Nivel 2: [20 23 23]
Nivel 2: [29 30 111]

Folhas (encadeadas):
[5 7] -> [10 10 12] -> [12 17] -> [20 23 23] -> [29 30 111] -> NULL
```

5:

```
===== MENU B+ TREE =====
1 - Inserir valores
2 - Buscar valor
3 - Remover valor
4 - Ver arvore
0 - Sair
Escolha: 3

Digite o valor para remover: 5

Valor 5 removido!

===== MENU B+ TREE =====
1 - Inserir valores
2 - Buscar valor
3 - Remover valor
4 - Ver arvore
0 - Sair
Escolha: 4

Arvore B+:
Nivel 0: [20]
Nivel 1: [10 12]
Nivel 2: [7 10]
Nivel 2: [10 12]
Nivel 2: [12 17]
Nivel 1: [29]
Nivel 2: [20 23 23]
Nivel 2: [29 30 111]

Folhas (encadeadas):
[7 10] -> [10 12] -> [12 17] -> [20 23 23] -> [29 30 111] -> NULL
```

7:

```
===== MENU B+ TREE =====
1 - Inserir valores
2 - Buscar valor
3 - Remover valor
4 - Ver arvore
0 - Sair
Escolha: 3

Digite o valor para remover: 7

Valor 7 removido!

===== MENU B+ TREE =====
1 - Inserir valores
2 - Buscar valor
3 - Remover valor
4 - Ver arvore
0 - Sair
Escolha: 4

Arvore B+:
Nivel 0: [12 20 29]
Nivel 1: [10 10 12]
Nivel 1: [12 17]
Nivel 1: [20 23 23]
Nivel 1: [29 30 111]

Folhas (encadeadas):
[10 10 12] -> [12 17] -> [20 23 23] -> [29 30 111] -> NULL
```

Durante a execução, o programa demonstra:

- Divisões sucessivas de folhas
- Promoções corretas ao nível interno
- Encadeamento de folhas após cada operação
- Operações de borrow e merge funcionando
- Ajustes completos da raiz

## Conclusão

A Árvore B+ demonstrou, na prática, sua eficiência como estrutura de indexação, mantendo balanceamento automático e baixo nível de profundidade - essenciais para sistemas de armazenamento real.

```
===== MENU B+ TREE =====
1 - Inserir valores
2 - Buscar valor
3 - Remover valor
4 - Ver arvore
0 - Sair
Escolha: 0

Encerrando...
```